AssemblyFlow: Pipelined Execution Analyzer

Subject Micro Project Report: Computer Organization & Architecture (COA)

Student Name:	Ashik Ameen	**Subject:**	Computer Organization & Architecture (COA)
Project:	AssemblyFlow	**Academic Term:**	October 2026
Live Demo:	https://micro-project-a338.onrender.com/	**GitHub Repo:**	https://github.com/Ashik9500/Micro_project.git

1. Executive Summary & Problem Statement

Pipelining is the core microarchitectural technique employed in modern microprocessors to maximize instruction throughput by overlapping execution phases. In **AssemblyFlow**, the **Computer Organization and Architecture (COA)** module functions as **Stage 3**. It simulates cycle-by-cycle execution of verified assembly code across the classic **5-stage RISC pipeline** (IF, ID, EX, MEM, WB), evaluates **Read-After-Write (RAW)** data hazards, manages **split-phase register file timing**, models **stall bubble insertion**, and provides an interactive demonstration of **hardware data forwarding**.

2. Theoretical Foundations in Computer Architecture

2.1 Classic 5-Stage RISC Pipeline Model

The simulator implements the canonical five-stage in-order RISC microarchitecture:
• **IF (Instruction Fetch):** Fetches instruction from instruction memory into the IF/ID pipeline register.
• **ID (Instruction Decode / Register Read):** Decodes opcode, reads source registers from the dual-port Register File, and checks for hazards.
• **EX (Execution / Effective Address):** The ALU executes arithmetic operations (ADD, SUB) or passes operands.
• **MEM (Memory Access):** Memory cache stage (immediate loads pass through to MEM/WB register).
• **WB (Write-Back):** Commits result to the destination register in the Register File.

2.2 Split-Phase Register File Timing

AssemblyFlow models **split-phase register file clocking**: register writes occur during the *first half of the clock cycle* (0 to T/2) in WB, while register reads occur during the *second half of the cycle* (T/2 to T) in ID. Thus, if instruction I1 is in WB in cycle C, instruction I2 can safely read the updated register in ID during the very same cycle C without needing an extra stall cycle.

2.3 Data Hazards: Stall Bubbles vs. Hardware Forwarding

A **Read-After-Write (RAW)** hazard arises when consumer instruction I2 requires a register written by producer I1:
• **Baseline Interlock (Forwarding Disabled):** I2 is held in ID until I1 commits to the register file in WB. Two NOP stall bubbles are injected into EX, delaying subsequent instructions.
• **Hardware Data Forwarding (Bypassing Enabled):** Dedicated multiplexers route ALU outputs from EX/MEM directly to EX stage ALU inputs, eliminating RAW stalls for ALU instructions (0 stall cycles).

3. Mathematical Performance Formulations

For N instructions executed across k = 5 pipeline stages:
• **Non-Pipelined Cycles:** $C_{non_pipelined} = N \times 5$
• **Ideal Pipelined Cycles:** $C_{ideal} = N + (k - 1) = N + 4$
• **Actual Cycles (with Stalls):** $C_{actual} = N + 4 + \Sigma \text{ Stalls}$
• **Cycles Per Instruction (CPI):** $CPI = C_{actual} / N$
• **Speedup (S):** $S = C_{non_pipelined} / C_{actual} = (5 \times N) / C_{actual}$

4. Experimental Benchmark: 3-Instruction RAW Dependency Chain

Benchmark program: LOAD R1, 10 → ADD R2, R1, R3 → SUB R4, R2, R5

Metric / Parameter	Forwarding Disabled (Default)	Forwarding Enabled	Microarchitectural Gain

Total Clock Cycles	11 CC	7 CC	**36.4% Cycle Reduction**
Injected Stall Bubbles	4 Stalls (2 for I2, 2 for I3)	0 Stalls	**100% Bubbles Eliminated**
Cycles Per Instruction (CPI)	3.67	2.33	**36.5% CPI Improvement**
Pipeline Speedup (vs Non-Pipelined)	1.36x	2.14x	**+57.4% Throughput Boost**

Timing Matrix with Forwarding Disabled (11 Cycles):

```
I1: [IF] [ID] [EX] [MEM] [WB]
I2: -- [IF] (STALL) (STALL) [ID] [EX] [MEM] [WB]
I3: -- -- -- -- [IF] (STALL) (STALL) [ID] [EX] [MEM] [WB]
```

Timing Matrix with Forwarding Enabled (7 Cycles):

```
I1: [IF] [ID] [EX] [MEM] [WB]
I2: -- [IF] [ID] [EX] [MEM] [WB] (Operand forwarded from I1 EX stage)
I3: -- -- [IF] [ID] [EX] [MEM] [WB] (Operand forwarded from I2 EX stage)
```

5. Conclusions & Learning Outcomes

The COA pipeline simulation engine models actual RISC microarchitectural execution with complete fidelity. Students can observe the direct physical consequences of data dependencies, verify the split-phase register file assumptions, and dynamically quantify the throughput gains delivered by hardware forwarding bypass paths.